SmartEduLoan

Software Requirements Specification (SRS)

A Full-Stack Student Loan Management System

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) provides a comprehensive description of the functional and non-functional requirements for the **SmartEduLoan** web application. It serves as the definitive reference for the development, testing, and validation of the system.

1.2 Scope

SmartEduLoan is a multi-role web platform that automates the student loan lifecycle — from online application submission, through credit score evaluation and administrative review, to bank verification and EMI repayment tracking. The system supports three distinct user roles (Student, Admin, Bank) with isolated portals and enforced access control.

1.3 Definitions

Term	Definition
JWT	JSON Web Token — a compact, stateless token used for secure user authentication across API requests
Credit Score	A numeric value automatically computed from a student's academic and financial profile data that determines loan eligibility
EMI	Equated Monthly Installment — a fixed, pre-calculated monthly repayment amount covering principal and interest
Mongoose ODM	Object Document Mapper library for Node.js that provides a schema-based interface to MongoDB
Multer	A Node.js middleware for handling multipart/form-data, used here for document file uploads
REST API	Representational State Transfer API — a stateless communication architecture used between the frontend and backend

---

2. Overall System Description

2.1 System Architecture

The application follows a classic **three-tier architecture**:

```
Browser (Student / Admin / Bank)
        │  HTTP Requests (REST)
        ▼
  Frontend — HTML + CSS + Vanilla JS     [port 3000]
        │  JSON over HTTP
        ▼
  Backend — Node.js + Express.js         [port 5001]
        │  Mongoose Queries
        ▼
  Database — MongoDB
```

The frontend is a static site served using `npx serve`. It communicates with the Express backend exclusively through REST API calls, with all protected routes validated via JWT middleware.

2.2 User Roles & Access Control

Role	Portal URL	Permissions
Student	`/student/dashboard`	Profile management, loan application, status tracking, EMI viewing
Admin	`/admin/dashboard`	Full application management, bank assignment, approve/reject authority

Bank	`/bank/dashboard`	View and verify only the applications assigned to them

---

3. Functional Requirements

3.1 Authentication Module

ID	Requirement	Details
FR-01	User registration	A user can register by providing their name, email, password, and role (student/admin/bank)
FR-02	User login	A registered user can log in using their email and password
FR-03	Token issuance	On successful login, the system generates and returns a signed JWT token
FR-04	Route protection	All non-public routes validate the JWT from the Authorization header using middleware
FR-05	Password security	All passwords are hashed using bcrypt before being stored in the database

3.2 Student Module

ID	Requirement	Details

FR-06	Profile creation	Students complete a structured profile including academic details (institution, course, year, percentage), financial information (income, existing loans, expenses), and address
FR-07	Credit score computation	The system automatically calculates a numeric credit score based on the student's submitted profile data
FR-08	Loan application	Students submit a loan application specifying the requested amount, repayment tenure, loan purpose, and any supporting documents
FR-09	Document upload	Students upload PDF or image files as supporting documents; files are handled via Multer and stored in the server's upload directory
FR-10	Loan history	Students can view all past and current loan applications with status labels
FR-12	Auto-rejection	The system automatically rejects any loan application where the computed credit score is below 600, with an appropriate rejection reason communicated to the student

3.3 Admin Module

ID	Requirement	Details
FR-13	Application list	Admin can view all loan applications across all students, with filter options for status (pending / approved / rejected / under review)
FR-14	Bank assignment	Admin can assign any application to a specific registered bank for verification
FR-15	Loan approval	Admin can approve an application; this triggers EMI schedule generation and updates the loan status
FR-16	Loan rejection	Admin can reject an application with a mandatory reason that is displayed to the student
FR-17	Bank management	Admin can view, add, and manage registered partner banks in the system
FR-18	Dashboard summary	Admin dashboard shows aggregated counts: total applications, pending, approved, and rejected

3.4 Bank Module

ID	Requirement	Details
FR-19	Filtered application view	Banks can only view the applications that have been specifically assigned to them by the admin

FR-20	Document access	Banks can access all documents and profile information attached to an assigned application for thorough verification
FR-21	Verification verdict	Banks submit a verification verdict (verified / flagged), which is recorded and available to the admin for informed decision-making

---

4. Data Models

4.1 User

```
User {
  name       : String   — Full name of the user
  email      : String   — Unique identifier for
authentication
  password   : String   — bcrypt hashed
password
  role       : Enum     — ['student', 'admin',
'bank']
}
```

4.2 StudentProfile

```
StudentProfile {
  userId       : ObjectId → User
  academicInfo : {
    institution : String,
    course      : String,
    year        : Number,
    percentage  : Number
  }
  financialInfo : {
    income        : Number,
    expenses      : Number,
    existingLoans : Number
  }
  address      : { street, city, state, pincode
}
  creditScore  : Number — computed
automatically
}
```

4.3 Loan

```
Loan {
  studentId       : ObjectId → User
  bankId          : ObjectId → Bank  (set on
assignment)
  amount          : Number           —
```

```
requested loan amount
  tenure          : Number           —
repayment duration in months
  purpose         : String           — stated
purpose of the loan
  status          : Enum ['pending',
'approved', 'rejected', 'under_review']
  rejectionReason : String           —
populated on rejection
  documents       : [String]         — file
paths to uploaded documents
  appliedAt       : Date
}
```

4.4 Bank

```
Bank {
  name    : String   — registered bank name
  userId  : ObjectId → User
  contact : String   — bank contact info
}
```

4.5 BankProduct

```
BankProduct {
  bankId       : ObjectId → Bank
  productName  : String
  interestRate : Number   — annual interest
rate (%)
  minTenure    : Number   — minimum repayment
term (months)
  maxTenure    : Number   — maximum repayment
term (months)
  maxAmount    : Number   — maximum eligible
loan amount
}
```

---

5. REST API Specification

Auth Routes — `/api/auth`

Method	Endpoint	Description	Auth Required
POST	`/register`	Register a new user account	No
POST	`/login`	Authenticate and receive a JWT token	No

Student Routes — `/api/student`

Method	Endpoint	Description	Auth Required
GET	`/profile`	Retrieve the student's current profile	Yes
POST	`/profile`	Create or update the student's profile	Yes
POST	`/loans/apply`	Submit a new loan application with documents	Yes
GET	`/loans`	Retrieve all loan applications for this student	Yes
GET	`/loans/:id`	Retrieve details of a specific loan	Yes

Admin Routes — `/api/admin`

Method	Endpoint	Description	Auth Required
GET	`/applications`	Fetch all loan applications with	Yes (Admin)

		optional filters	
PUT	`/applications/:id/approve`	Approve a loan and trigger EMI generation	Yes (Admin)
PUT	`/applications/:id/reject`	Reject a loan with a specified reason	Yes (Admin)
PUT	`/applications/:id/assign`	Assign a loan application to a registered bank	Yes (Admin)
GET	`/banks`	Fetch all registered bank partners	Yes (Admin)
POST	`/banks`	Register a new bank in the system	Yes (Admin)

Bank Routes — `/api/bank`

Method	Endpoint	Description	Auth Required
GET	`/applications`	Retrieve all applications assigned to this bank	Yes (Bank)
PUT	`/applications/:id/verify`	Submit a verification verdict for an application	Yes (Bank)

---

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	**Security**	All API routes except `/auth/register` and `/auth/login` must validate a JWT token — unauthenticated requests receive a 401 response
NFR-02	**Security**	Passwords are never stored in plain text; bcrypt hashing with an appropriate salt factor is mandatory
NFR-03	**Access Control**	Role-specific middleware must prevent students from accessing admin routes,

		and banks from accessing student or admin routes
NFR-04	**Performance**	API responses must be returned within 500ms under standard load conditions
NFR-05	**Usability**	The interface must function correctly in current versions of Chrome, Firefox, and Edge without requiring plugins or extensions
NFR-06	**Reliability**	Uploaded files must be validated for allowed MIME types and reasonable size limits before storage
NFR-07	**Maintainability**	Backend code must follow a clear MVC-like separation: models define schema, routes handle HTTP logic, and services encapsulate business logic
NFR-08	**Portability**	The system must run fully on a local machine with no mandatory cloud services or external infrastructure

---

7. System Constraints

- The frontend does not use any JavaScript framework — all interactivity is written in Vanilla JavaScript.
- The application targets a localhost development environment; no production deployment configuration is included.

- A single MongoDB instance handles all data — no sharding, replication, or clustering is configured.
- File uploads are stored on the local filesystem; no cloud object storage (e.g., AWS S3) is used.
- No real payment gateway is integrated; "Pay Now" in the EMI viewer is a UI interaction only.

---

8. Verification & Validation Plan

Test Case	Input / Action	Expected Outcome
Duplicate registration	Register with an already-used email	System returns HTTP 400 with a clear error message
Wrong password login	Submit incorrect password on login form	System returns HTTP 401 Unauthorized
Cross-role route access	Student attempts to call `/api/admin/applications`	System returns HTTP 403 Forbidden
Low credit score application	Apply with a profile that yields credit score < 600	Loan is auto-rejected with reason recorded

Successful loan approval	Admin approves a valid application	Loan status updates to 'approved'; EMI rows generated and stored
Bank assignment	Admin assigns loan to bank B	Bank B sees the application in their portal; other banks do not
EMI schedule accuracy	Check EMI amount after approval	EMI = f(principal, rate, tenure); values match expected calculation
Document upload	Upload a valid PDF via the apply form	File stored in the uploads directory; path recorded in the loan